CS6650 Assignment 2 — Design Document: Distributed WebSocket Chat System

Git Repository

https://github.com/StarfishJ/Distributed-System-Assignment-1.git

1. System Architecture

Our architecture utilizes a decoupled, event-driven pattern to achieve horizontal scalability. Servers handle WebSocket I/O via Netty, while a standalone Consumer layer manages global broadcast logic via RabbitMQ.

[Diagram 1: System Overview]

```
graph TD
    Client["Java Client"] --> ALB["AWS Application Load Balancer"]
    ALB --> S1["Server-v2 Node (+ optional duplicate nodes S2, S3...)"]
    S1 --> RMQ_T[("RabbitMQ: Topic chat.exchange")]
    RMQ_T --> CONS["Standalone Consumer"]
    CONS --> RMQ_F[("RabbitMQ: Fan-out broadcast.exchange")]
    RMQ_F --> |Push| S1
    S1 --> |WebSocket| Client
```

---

2. Message Flow Sequence

Each message undergoes a "Two-Hop" delivery to ensure all users in a room receive it, regardless of which server they are connected to.

```
sequenceDiagram
    participant C as Client
    participant S as Server-v2
    participant RMQ as RabbitMQ
    participant CONS as Consumer

    C->>S: "1. Send WebSocket Message"
    S->>RMQ: "2. Publish to room.{id} (Topic)"
    S-->>C: "3. ACK (QUEUED)"
    RMQ->>CONS: "4. Deliver to Queue"
    CONS->>RMQ: "5. Broadcast to Fan-out"
    RMQ-->>S: "6. Push to local nodes"
    S->>C: "7. Deliver to WebSocket Session"
```

---

3. Queue Topology & Consumer Threading

Queue Design

- **Topic Exchange (`chat.exchange`)**: Handles ingress messages. Routing key: `room.{roomId}`.
- **20 Room Queues**: Unlimited rooms are mapped to 20 durable queues using a **Hashing Strategy**. (`room.{id % 20}`).
- **Room ID Preservation**: Crucially, the original roomId is preserved within the message payload. When the Consumer broadcasts the message, server nodes use this payload roomId to deliver messages to the correct WebSocket sessions, allowing any number of rooms to coexist without collision.
- **Fan-out Exchange (`broadcast.exchange`)**: Distributes valid messages to all server nodes.

Consumer Threading Model

- **Thread-to-Queue Affinity**: The Consumer uses a pool of 20 concurrent threads.
- **Isolation**: Each thread is pinned to a specific room queue (`prefetch=1`). This guarantees **strict message ordering** within each room while allowing parallel processing across rooms.

---

4. Load Balancing & Failure Handling

Load Balancer Configuration

- **AWS ALB**: Handles WebSocket protocol upgrade and terminates TLS (if applicable).
- **Sticky Sessions**: Enabled via duration-based cookies to ensure a client stays on the same server node for the duration of the stateful WebSocket handshake.

Failure Handling & Resilience

Circuit Breaker & Backpressure -implemented a robust fail-fast mechanism using **Resilience4j**. If RabbitMQ becomes unresponsive (e.g., during AWS EBS throttling), the **Circuit Breaker** trips to the `OPEN` state.

- **Server Logic**: Instead of dropping messages or blocking, the server immediately responds with `{"status":"ERROR","message":"SERVER_BUSY"}`.

- **Client Resilience**: The Java client detects this specific error and performs an `offerFirst()` re-queue, ensuring **at-least-once delivery** despite physical hardware limitations.

- **Deduplication**: The Consumer uses a **Caffeine Cache** (LRU) to drop duplicate `UUIDs` that may arise during network retries or RabbitMQ redeliveries.

- **Graceful Shutdown**: All nodes use Spring's graceful shutdown (30s timeout) to ensure in-flight AMQP ACKs are processed before the process exits.

Performance Results

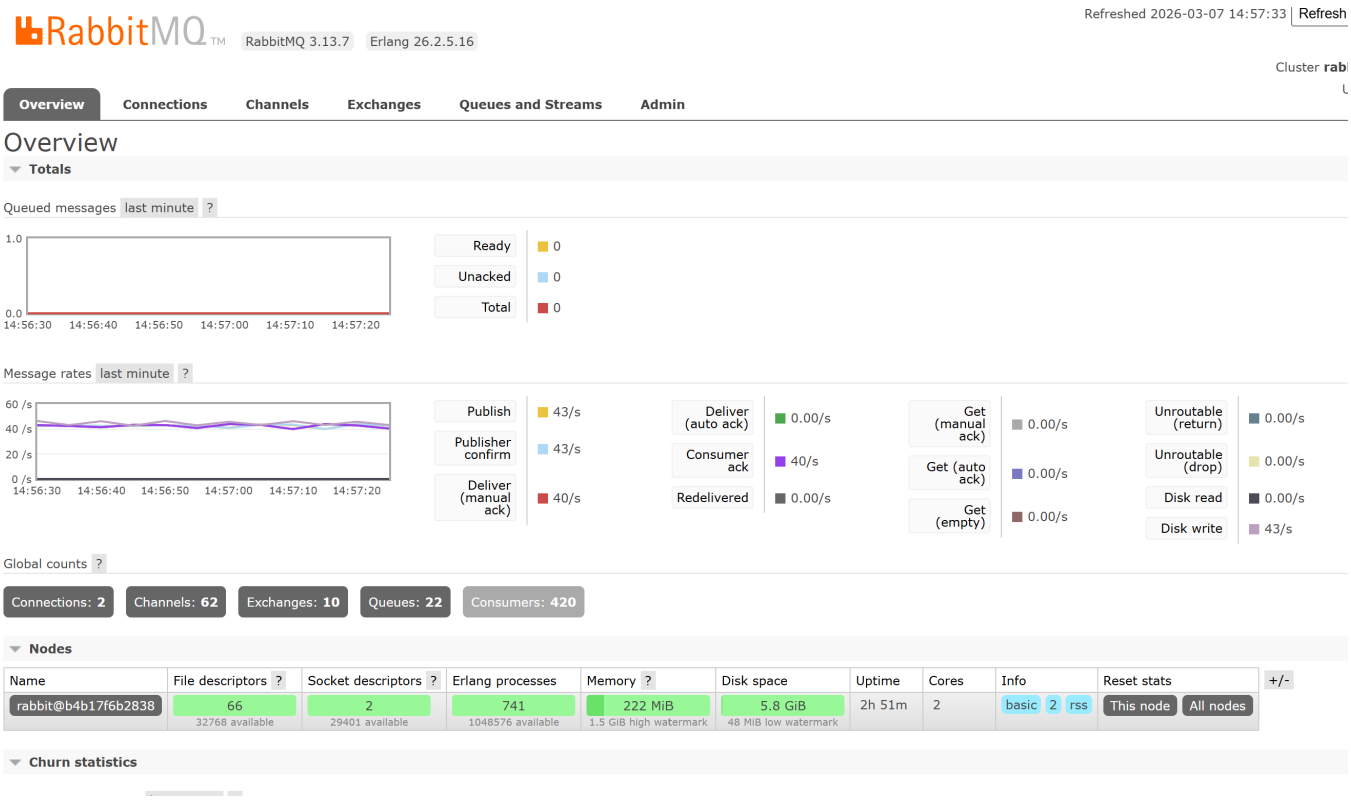
Single Server Node Baseline

- **Aggregate Throughput**: ~3,506 msg/s
- **Peak Rate**: ~4,800 msg/s
- **Success Rate**: **100.00%** (500,000 messages)

- **Survival**: Successfully survived AWS EBS I/O credit exhaustion via Backpressure logic.

```
[Main] progress: 89600 / 500000 sent (ok=89600, fail=0) — 4221.6 msg/s (bucket: 4320.0 msg/s)
[Main] progress: 131800 / 500000 sent (ok=131800, fail=0) — 4220.3 msg/s (bucket: 4220.0 msg/s)
[Main] progress: 174000 / 500000 sent (ok=174000, fail=0) — 4221.7 msg/s (bucket: 4220.0 msg/s)
[Main] progress: 217200 / 500000 sent (ok=217200, fail=0) — 4240.3 msg/s (bucket: 4320.0 msg/s)
[Main] progress: 259200 / 500000 sent (ok=259200, fail=0) — 4234.0 msg/s (bucket: 4200.0 msg/s)
[Main] progress: 301000 / 500000 sent (ok=301000, fail=0) — 4226.1 msg/s (bucket: 4180.0 msg/s)
[Main] progress: 344100 / 500000 sent (ok=344100, fail=0) — 4236.9 msg/s (bucket: 4310.0 msg/s)
[Main] progress: 384717 / 500000 sent (ok=384717, fail=0) — 4217.5 msg/s (bucket: 4061.7 msg/s)
[Main] progress: 421102 / 500000 sent (ok=421102, fail=0) — 4160.3 msg/s (bucket: 3638.5 msg/s)
[Main] progress: 453042 / 500000 sent (ok=453042, fail=0) — 4073.6 msg/s (bucket: 3194.0 msg/s)
[Main] progress: 483062 / 500000 sent (ok=483062, fail=0) — 3985.0 msg/s (bucket: 3002.0 msg/s)
[Main] progress: 498130 / 500000 sent (ok=498130, fail=0) — 3796.3 msg/s (bucket: 1506.8 msg/s)
[Main] Putting poison (sent so far: 500000)...
[Main] Poison pills added. Waiting for workers to finish...
[Main] progress: 500000 / 500000 sent (ok=500000, fail=0) — 3540.7 msg/s (bucket: 187.0 msg/s)
[Main] Finished.

------------------------------------------------------------
🚀 ASSIGNMENT 2 PERFORMANCE REPORT SUMMARY
------------------------------------------------------------
Throughput                : 3506.95 msg/s
Total Messages (Success)  : 500000
P95 Latency               : 173 ms
Connection Failures       : 0
Success Rate              : 100.00%
------------------------------------------------------------
Room Uniformity Check: R1=173.8 R10=175.2 R20=175.8 msg/s
P95 Latency               : 173 ms
Connection Failures       : 0
Success Rate              : 100.00%
------------------------------------------------------------
Room Uniformity Check: R1=173.8 R10=175.2 R20=175.8 msg/s
[Main] Throughput data exported to: results/throughput_over_time.csv (14 data points)
[Metrics] CSV writer shutdown complete. Queue size: 0
[Main] Per-message metrics CSV export completed (async write).
```



Overview | Connections | Channels | Exchanges | Queues and Streams | Admin

Connections

All connections (2)

Pagination

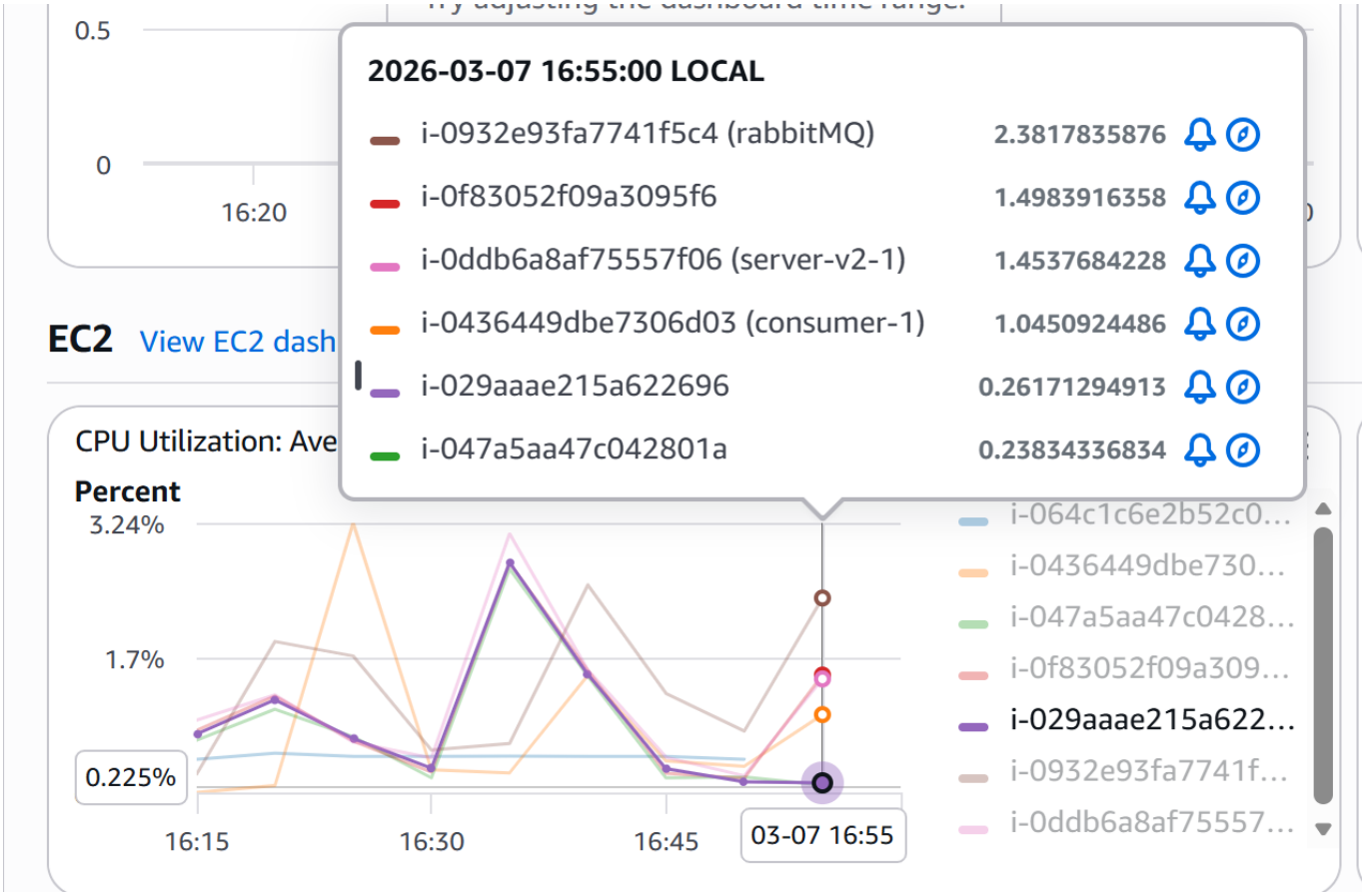
Page 1 of 1 - Filter: Regex ?

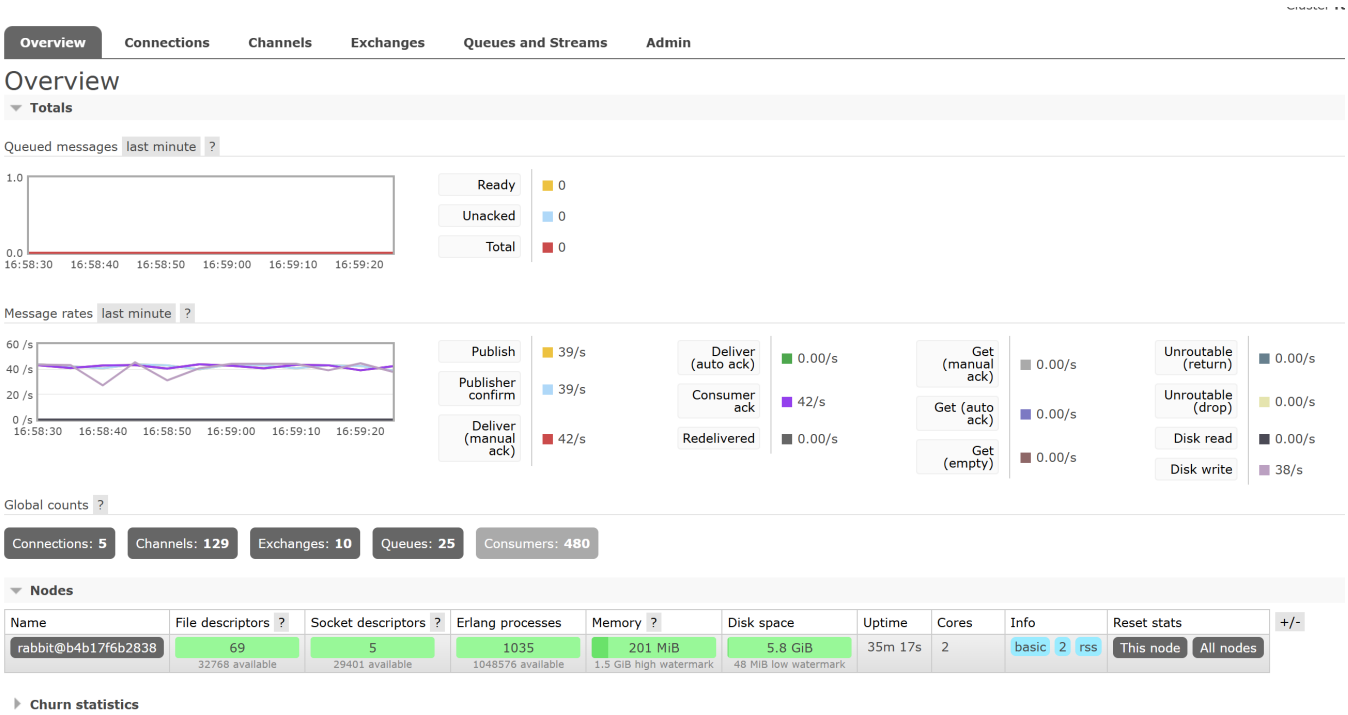
Overview			Details			Network		+/-
Name	**User name**	**State**	**SSL / TLS**	**Protocol**	**Channels**	**From client**	**To client**	
172.31.52.142:37272 rabbitConnectionFactory#34208baa:0	guest	running		AMQP 0-9-1	42	1005 KiB/s	890 B/s	
172.31.61.8:35166 rabbitConnectionFactory#3cd26422:0	guest	running		AMQP 0-9-1	20	857 B/s	969 KiB/s	

Two Server Nodes

```
[Main] progress: 46400 / 500000 sent (ok=46400, fail=0) — 4137.3 msg/s (bucket: 4640.0 msg/s)
[Main] progress: 89600 / 500000 sent (ok=89600, fail=0) — 4223.0 msg/s (bucket: 4320.0 msg/s)
[Main] progress: 132000 / 500000 sent (ok=132000, fail=0) — 4227.2 msg/s (bucket: 4240.0 msg/s)
[Main] progress: 173600 / 500000 sent (ok=173600, fail=0) — 4212.0 msg/s (bucket: 4160.0 msg/s)
[Main] progress: 216800 / 500000 sent (ok=216800, fail=0) — 4233.1 msg/s (bucket: 4320.0 msg/s)
[Main] progress: 259200 / 500000 sent (ok=259200, fail=0) — 4233.8 msg/s (bucket: 4240.0 msg/s)
[Main] progress: 300800 / 500000 sent (ok=300800, fail=0) — 4223.6 msg/s (bucket: 4160.0 msg/s)
[Main] progress: 344400 / 500000 sent (ok=344400, fail=0) — 4240.4 msg/s (bucket: 4360.0 msg/s)
[Main] progress: 385222 / 500000 sent (ok=385222, fail=0) — 4223.0 msg/s (bucket: 4082.2 msg/s)
[Main] progress: 481595 / 500000 sent (ok=481595, fail=0) — 3972.8 msg/s (bucket: 2769.9 msg/s)
[Main] progress: 496716 / 500000 sent (ok=496716, fail=0) — 3785.2 msg/s (bucket: 1512.1 msg/s)
[Main] Putting poison (sent so far: 500000)...
[Main] Poison pills added. Waiting for workers to finish...
[Main] Finished.

------------------------------------------------------------
🚀 ASSIGNMENT 2 PERFORMANCE REPORT SUMMARY
------------------------------------------------------------
Throughput                : 3587.08 msg/s
Total Messages (Success)  : 500000
Total Runtime             : 139.39 sec
Mean Latency (E2E ACK)    : 148.50 ms
P95 Latency               : 160 ms
Connection Failures       : 0
Success Rate              : 100.00%
------------------------------------------------------------
Room Uniformity Check: R1=179.5 R10=178.1 R20=179.5 msg/s
[Main] Throughput data exported to: results/throughput_over_time.csv (13 data points)
[Metrics] CSV writer shutdown complete. Queue size: 0
[Main] Per-message metrics CSV export completed (async write).
```



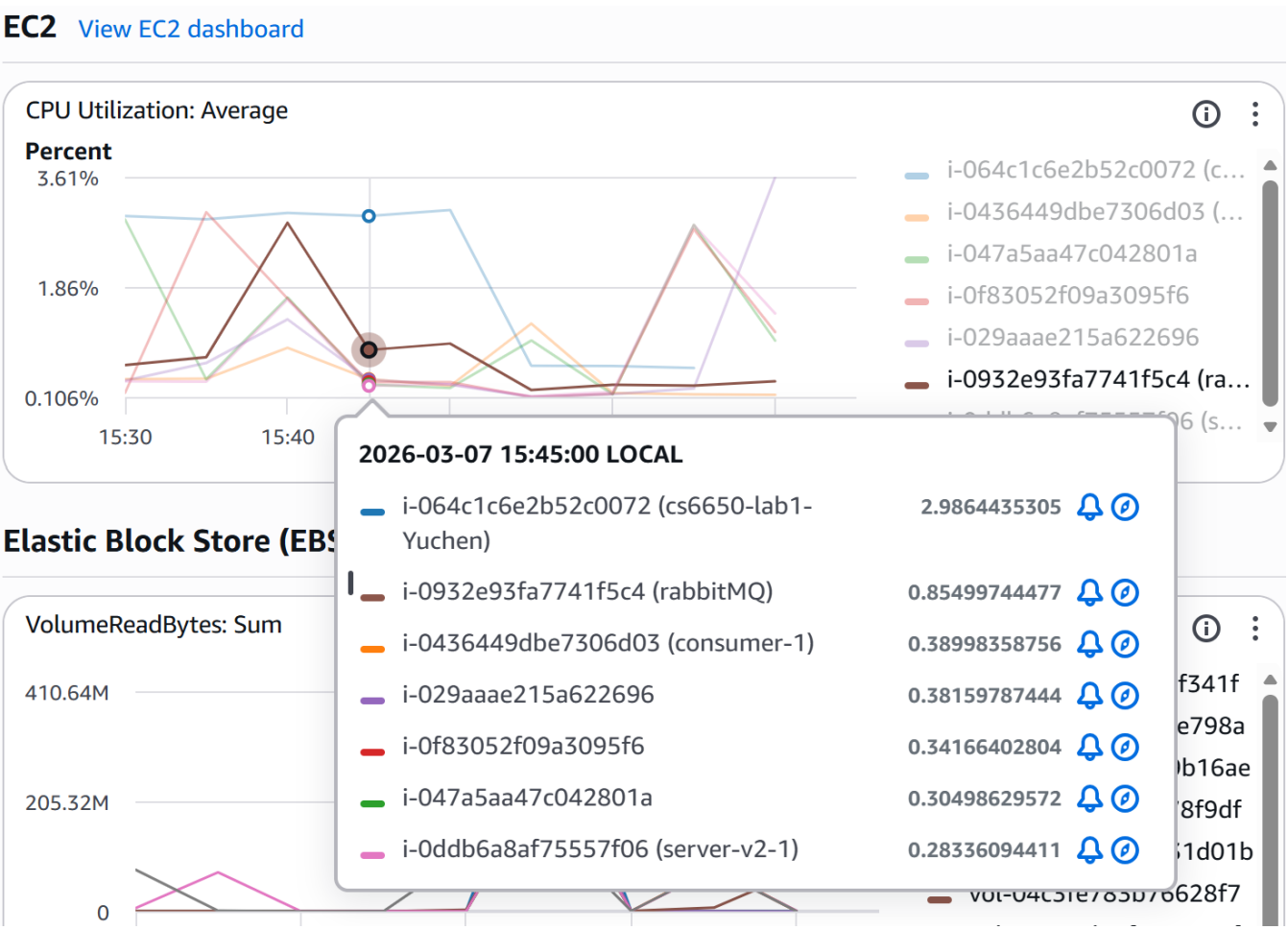


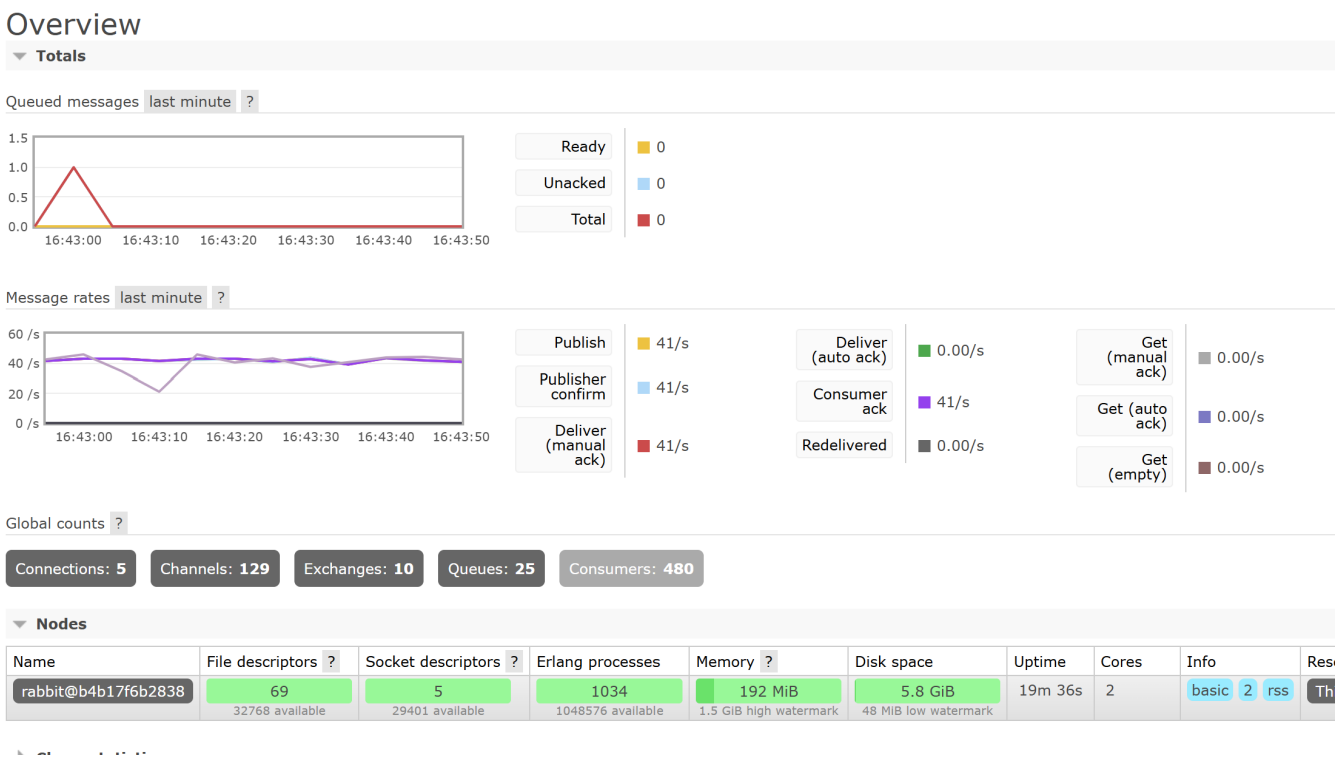
The throughput increased from 3506 msg/s to 3587 msg/s, which is an improvement of about 2.3%. The latency decreased from 206.39 ms to 149.87 ms, which is an improvement of about 27.4%.

Four Server Nodes

```
PS D:\6650\assignment 1\client\client_part2> mvn clean package -DskipTests
>> java -jar target/chat-client-part2-0.0.1-SNAPSHOT.jar http://chat-system-alb-396116245.us-east-1.el
ws.com 500000
>>
[Main] progress: 217200 / 500000 sent (ok=217200, fail=0) — 4241.3 msg/s (bucket: 4320.0 msg/s)
[Main] progress: 259500 / 500000 sent (ok=259500, fail=0) — 4239.4 msg/s (bucket: 4230.0 msg/s)
[Main] progress: 301600 / 500000 sent (ok=301600, fail=0) — 4235.3 msg/s (bucket: 4210.0 msg/s)
[Main] progress: 344400 / 500000 sent (ok=344400, fail=0) — 4240.8 msg/s (bucket: 4280.0 msg/s)
[Main] progress: 386201 / 500000 sent (ok=386201, fail=0) — 4234.1 msg/s (bucket: 4180.1 msg/s)
[Main] progress: 420961 / 500000 sent (ok=420961, fail=0) — 4159.2 msg/s (bucket: 3476.0 msg/s)
[Main] progress: 453501 / 500000 sent (ok=453501, fail=0) — 4077.8 msg/s (bucket: 3254.0 msg/s)
g/s (bucket: 2916.3 msg/s)
[Main] progress: 497691 / 500000 sent (ok=497691, fail=0) — 3793.1 msg/s (bucket: 1502.7 msg/s)
[Main] Putting poison (sent so far: 500000)...
[Main] Poison pills added. Waiting for workers to finish...
[Main] Finished.

------------------------------------------------------------
🚀 ASSIGNMENT 2 PERFORMANCE REPORT SUMMARY
------------------------------------------------------------
Throughput                : 3585.03 msg/s
Total Messages (Success)  : 500000
Total Runtime             : 139.47 sec
Mean Latency (E2E ACK)    : 150.40 ms
P95 Latency               : 167 ms
Connection Failures       : 0
Success Rate              : 100.00%
------------------------------------------------------------
Room Uniformity Check: R1=178.5 R10=177.6 R20=181.2 msg/s
[Main] Throughput data exported to: results/throughput_over_time.csv (13 data points)
[Metrics] CSV writer shutdown complete. Queue size: 0
[Main] Per-message metrics CSV export completed (async write).
```





Performance improvement analysis: The throughput is 3585 msg/s, which is virtually identical to the 2-node setup (3587 msg/s). This confirms that the system's performance is currently capped by **RabbitMQ's Disk I/O (EBS credits)** on the single-node broker, rather than the number of server nodes. However, the 4-node configuration provides significantly higher CPU headroom and resilience.

Configuration Detail

Queue configuration parameters

- **Room Queues**: 20 durable physical queues (`room.1` - `room.20`) used for sharding logic rooms via hashing (`roomId % 20`).
- **TTL (Time To Live)**: 5,000ms (Messages expire after 5 seconds to prevent memory overflow)
- **Max Length**: 1,000 messages per queue (Drop-head overflow policy)
- **Dead Letter Exchange**: `chat.dlx` for failed/expired messages.
- **Persistence**: Messages are marked persistent for durability.

Consumer configuration

- **Concurrency**: 20 (Fixed 1 thread per Room queue to ensure strict ordering)
- **Prefetch Count**: 1 (Ensures fair distribution and prevents message bunching)
- **Acknowledge Mode**: Manual (ACK sent only after successful processing/broadcast)
- **Idempotency**: Caffeine cache (10 min expiry) to deduplicate broadcast IDs.

ALB settings

- **Idle Timeout**: 4,000 seconds (To maintain long-lived WebSocket connections)
- **Stickiness**: Enabled (Load balancer generated cookie, duration: 86,400s)
- **Protocol**: HTTP (Port 80) mapped to backend 8080.
- **Health Check**: `GET /health` (Interval: 30s, Timeout: 5s, Threshold: 2)

Instance types used

- **Server Nodes**: 1 x `t3.medium` + optional `t3.micro` nodes
- **Consumer Node**: 1 x `t3.small`
- **RabbitMQ Node**: 1 x `t3.medium`
- **Client**: Running on local performance analysis node